DOCENTE	Wolfgang Cecchin
UNITA' FORMATIVA	Linux Server
ARGOMENTO	Shell avanzata

Shell avanzata

Reindirizzamento, processi e operatori

1. File descriptor e reindirizzamento

Cos'è un file descriptor

In Linux, tutto è un file: file di testo, directory, dispositivi hardware, socket di rete, la tastiera, lo schermo. Il kernel gestisce tutto tramite la stessa astrazione.

Quando un processo vuole comunicare con qualcosa — leggere dalla tastiera, scrivere sullo schermo, aprire un file — non lo fa direttamente. Chiede al kernel di aprire un canale, e il kernel gli restituisce un numero intero che rappresenta quel canale. Quel numero è il file descriptor (fd).

Puoi immaginarlo come il numero di un tavolo al ristorante: il cameriere (kernel) sa a quale tavolo portare il cibo, tu (il processo) usi solo il numero — non ti preoccupi di dove si trova fisicamente il tavolo.

```
processo → "voglio scrivere sul canale 1" → kernel → schermo
processo → "voglio scrivere sul canale 1" → kernel → file.txt   ← dopo
reindirizzamento
```

Il processo scrive sempre sullo stesso numero. È il kernel che decide dove quel numero punta davvero.

I tre canali aperti automaticamente

Ogni processo, nel momento in cui viene avviato, riceve già tre file descriptor aperti, ereditati dalla shell:

FD	Nome	Direzione	Collegato a (default)
0	stdin	ingresso	tastiera
1	stdout	uscita	terminale
2	stderr	uscita errori	terminale

stdout e stderr puntano entrambi al terminale, ma sono canali separati. Questo è fondamentale: permette di mandare l'output normale da una parte e gli errori da un'altra, senza mescolarli.

Perché si chiama "descriptor" e non "file"

Il file descriptor non è il file — è solo un indice nella tabella dei file aperti del processo. Ogni processo ha la propria tabella. Modificare dove punta l'fd 1 di un processo non cambia nulla per gli altri processi.

Il reindirizzamento (`>`, `2>`, `&>`) non fa altro che cambiare la destinazione di questi indici prima che il processo parta. Il processo non sa nulla di questo: continua a scrivere su `1` e `2` come sempre.

Reindirizzamento di stdout (1)

```
# Salva l'elenco dei file di configurazione su file
ls -l /etc > elenco.txt

# Aggiunge in coda invece di sovrascrivere
ls -l /var/www >> elenco.txt

# Equivalente esplicito con fd 1
ls -l /etc 1> elenco.txt

# Caso pratico: salvare il risultato di una query MySQL su file
mysql -u root -p fotodb -e "SELECT * FROM clienti;" > clienti.txt
```

Reindirizzamento di stderr (2)

```
# find genera errori su directory senza permessi — li separiamo
find / -name "*.conf" > risultati.txt 2> errori.log

# Cerca file PHP ignorando silenziosamente le directory protette
find /var/www -name "*.php" 2> /dev/null

# grep: solo i match su stdout, eventuali errori su file separato
grep -r "password" /etc 2> /tmp/grep_errori.log
```

Reindirizzare entrambi con 2>&1

La sintassi si legge pezzo per pezzo:

```
2  >  &1
^  ^  ^^
│  │  │└─ il numero del file descriptor 1 (stdout)
│  │  └── & significa "questo è un fd, non un nome file"
│  └───── operatore di reindirizzamento
└──────── il file descriptor 2 (stderr)
```

Traduzione letterale: **"punta fd 2 verso lo stesso posto dove punta fd 1"**.

Il `&` davanti all'`1` è necessario per disambiguare: senza, `2>1` significherebbe "scrivi stderr su un file chiamato letteralmente `1`".

Perché l'ordine conta

La shell legge il reindirizzamento da sinistra a destra. 2>&1 significa "copia la destinazione attuale di fd 1" — non "tieni fd 2 sempre sincronizzato con fd 1".

```
# CORRETTO: prima sposta stdout su file, poi aggancia stderr a stdout
find / -name "*.conf" > tutto.log 2>&1
```

```
# passo 1 — dopo > tutto.log:   fd1 → tutto.log,  fd2 → terminale
# passo 2 — dopo 2>&1:          fd1 → tutto.log,  fd2 → tutto.log  ✓



# SBAGLIATO: 2>&1 viene letto prima, quando stdout punta ancora al terminale
find / -name "*.conf" 2>&1 > tutto.log

# passo 1 — dopo 2>&1:          fd1 → terminale,  fd2 → terminale  ← copia fd1 com'è ora
# passo 2 — dopo > tutto.log:   fd1 → tutto.log,  fd2 → terminale  ✗ stderr rimane sul terminale



# Sintassi moderna equivalente (solo bash)
find / -name "*.conf" &> tutto.log

# Sintassi classica (funziona su sh, dash, bash — preferibile negli script)
find / -name "*.conf" > tutto.log 2>&1
```

Esempi pratici

```
# Backup MySQL: output e errori nello stesso log
mysqldump -u root -p fotodb > /var/log/backup.log 2>&1

# Aggiornamento pacchetti silenzioso, tutto su log
apt-get upgrade > /var/log/upgrade.log 2>&1

# Esecuzione silenziosa — scarta tutto
apt-get update &> /dev/null
```

Reindirizzamento di stdin (0)

```
# MySQL legge il dump da file invece che dalla tastiera
mysql -u root -p fotodb < dump.sql

# Invia il contenuto di un file come corpo di una mail
mail -s "Report giornaliero" admin@studio.it < /var/log/backup.log
```

/dev/null — il cestino

```
# Esegui silenziosamente, nessun output
apt-get update &> /dev/null
```

2. Operatori di controllo

; — esecuzione sequenziale (incondizionata)

```
apt-get update ; apt-get upgrade
# Il secondo gira sempre, anche se il primo fallisce
```

```
# Backup e pulizia: la pulizia parte sempre, indipendentemente dal backup
mysqldump -u root -p fotodb > backup.sql ; rm -f /tmp/export_temp.csv
```

&& — AND logico

Il secondo comando viene eseguito solo se il primo ha restituito codice 0.

```
# Crea la directory e solo se riesce, entra dentro
mkdir /var/www/nuovosito && cd /var/www/nuovosito

# Installa nginx e solo se riesce, avvia il servizio
apt-get install -y nginx && systemctl start nginx

# Importa il dump solo se il backup è andato a buon fine
mysqldump -u root -p fotodb > backup.sql && mysql -u root -p fotodb_test <
backup.sql
```

|| — OR logico

Il secondo comando viene eseguito solo se il primo ha restituito codice diverso da 0.

```
# Entra nella directory o, se non esiste, la crea
cd /var/www/html || mkdir -p /var/www/html

# Verifica connessione al DB o stampa errore e termina lo script
mysql -u root -p -e "SELECT 1;" &>/dev/null || { echo "DB non raggiungibile";
exit 1; }
```

Combinazione && e ||

```
# Simula un if/else in una riga
ping -c1 8.8.8.8 &>/dev/null && echo "Rete OK" || echo "Rete non raggiungibile"

# Verifica se nginx è attivo
systemctl is-active nginx &>/dev/null && echo "nginx in esecuzione" || echo
"nginx fermo"
```

! — negazione del codice di uscita

```
# Crea il file di config solo se non esiste già
[ ! -f /etc/nginx/conf.d/studio.conf ] && cp /etc/nginx/conf.d/default.conf
/etc/nginx/conf.d/studio.conf
```

2b. $? — il codice di uscita

Ogni comando restituisce un numero intero quando termina, chiamato exit code. È il valore che && e || controllano internamente.

Valore	Significato
0	successo

≠ 0	errore

```
ls /etc
echo $?        # → 0

ls /cartella_inesistente
echo $?        # → 2

grep "root" /etc/passwd
echo $?        # → 0  (trovato)

grep "zzzz" /etc/passwd
echo $?        # → 1  (nessun match — grep usa 1, non 2)

mysql -u root -pwrong -e "SELECT 1;" &>/dev/null
echo $?        # → 1  (autenticazione fallita)
```

> ⚠ *$? viene sovrascritto ad ogni comando, incluso echo. Va letto subito dopo il comando che interessa.*

Nei casi reali: salvarlo in una variabile

```
mysqldump -u root -p fotodb > backup.sql
ESITO=$?

[ $ESITO -eq 0 ] && echo "Backup OK" || echo "Backup fallito (codice: $ESITO)"
```

Exit code più comuni

Codice	Significato
0	successo
1	errore generico
2	uso errato del comando
127	comando non trovato
130	interrotto con Ctrl+C

3. Processi in background

& — avvio in background

```
# Il terminale torna subito disponibile
php artisan queue:work &
# [1] 4821  ← numero job e PID
```

jobs — lista processi in background della sessione

```
jobs
# [1]+  Running    php artisan queue:work &
# [2]-  Stopped    vim file.txt
```

fg e bg

```
fg          # porta in foreground il job corrente (+)
fg %2       # porta in foreground il job numero 2

bg          # riprende in background il job sospeso
bg %2       # riprende in background il job 2
```

Ctrl+Z — sospende il processo in foreground

```
[Ctrl+Z]
# [1]+  Stopped    ping google.com

bg %1       # lo riprendi in background
```

kill su job

```
kill %1     # termina il job 1 (SIGTERM)
kill -9 %1  # termina forzatamente (SIGKILL)
```

> *I job in background sono legati alla sessione shell corrente: se chiudi il terminale, vengono terminati.*

4. nohup — processi persistenti

`nohup` permette a un processo di sopravvivere alla chiusura del terminale, ignorando il segnale `SIGHUP`.

```
# Avvio base — output automatico su nohup.out
nohup php artisan queue:work &

# Con log personalizzato
nohup python3 script.py > /var/log/myscript.log 2>&1 &

# Scartando tutto l'output
nohup ./backup.sh &> /dev/null &
```

Differenza chiave: & vs nohup &

	comando &	nohup comando &
Terminale chiuso	processo terminato	processo continua
Output	sul terminale	nohup.out (default)

Segnale SIGHUP	ricevuto	ignorato

Verificare che il processo stia girando

```
ps aux | grep queue:work | grep -v grep
```

5. Esercizio — Scenario: Studio fotografico

Lo studio Luce & Scatto ti chiede di preparare uno script di manutenzione notturna per il proprio server Linux. Lo script deve:

a) Creare la directory /var/log/studio/ se non esiste già, e interrompersi con un messaggio di errore se la creazione fallisce.

b) Esportare la tabella sessioni del database fotodb su /tmp/sessioni_export.csv, salvando eventuali errori in /var/log/studio/errori.log. Verificare il codice di uscita e stampare un messaggio di esito.

c) Eseguire find /var/www/fotografie -name "*.tmp" salvando i risultati su /var/log/studio/tmp_trovati.log e gli errori su /var/log/studio/errori.log (in aggiunta, non sovrascrivendo).

d) Avviare lo script in background con nohup, in modo che continui a girare dopo il logout, con tutto l'output su /var/log/studio/manutenzione.log.

e) Verificare con un solo comando che lo script sia effettivamente in esecuzione.

Riepilogo

```
Reindirizzamento:
  >           stdout su file (sovrascrive)
  >>          stdout su file (aggiunge)
  2>          stderr su file
  2>&1        stderr → dove punta stdout (ordine: prima >, poi 2>&1)
  &>          stdout + stderr su file (solo bash)
  <           stdin da file

Operatori:
  ;           esegui sempre in sequenza
  &&          esegui solo se $? = 0
  ||          esegui solo se $? ≠ 0
  !           nega il codice di uscita

$?:
  0           successo
  ≠ 0         errore (salvarlo subito in una variabile)

Background:
  cmd &       avvia in background
  Ctrl+Z      sospendi in foreground
  jobs        lista job sessione
  fg %n       porta in foreground
  bg %n       riprendi in background
  kill %n     termina job
```

```
nohup:
  nohup cmd &                 persiste dopo logout
  nohup cmd > log 2>&1 &      con log personalizzato
```